

The Krasnosel'skiĭ-Mann Iterative Method

Chapter 7: Relaxation Parameters of the Krasnosel'skiĭ-Mann Iteration

Based on Dong, Cho, He, Pardalos & Rassias

July 13, 2026

Table of Contents

- 1 Notation Recap and New Concepts
- 2 7.0 Why Relaxation Parameters Matter: A Short History
- 3 7.1 The Approximate Optimal Relaxation Sequence
- 4 7.2 Variants via Variational Inequality Projection Methods
- 5 7.3 The Residual Algorithm
- 6 Chapter Summary

Notation You'll Need / Recap I

The Krasnosel'skiĭ-Mann (KM) iteration (recap). For a nonexpansive operator $T : \mathcal{H} \rightarrow \mathcal{H}$ with $\text{Fix}(T) \neq \emptyset$,

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T(x_n), \quad n \geq 0,$$

where λ_n is the **relaxation parameter**. Three regimes: *underrelaxed* if $\lambda_n \leq 1$, *unrelaxed* if $\lambda_n \equiv 1$, *overrelaxed* if $\lambda_n \geq 1$. This chapter is entirely about *how to choose* $\{\lambda_n\}$.

What “optimal” means here. Fix an iterate x_n and a fixed point $p \in \text{Fix}(T)$. Among *all* possible choices of $\lambda_n \in \mathbb{R}$, exactly one makes the *next* iterate x_{n+1} as close as possible to p (in norm). Call that value the **n -th optimal relaxation parameter** — plainly, “the best possible single step, in hindsight, if you already knew where you were headed.” It is a one-step-lookahead notion, not a statement about the whole trajectory.

Notation You'll Need / Recap II

The catch

Computing the optimal λ_n requires knowing a fixed point p *before* the iteration has found one — information you do not have in practice. Every formula in this chapter is either (a) an idealized optimal parameter that depends on the unknown p (useful only for analysis / benchmarking), or (b) a computable *approximation* that replaces p by quantities you can actually evaluate along the way (e.g. $T(x_n)$, $T(T(x_n))$), or (c) an adaptive rule driven by an observable error measure.

Notation You'll Need / Recap III

The variational inequality (VI) problem — plain language. Let $C \subseteq \mathcal{H}$ be closed and convex, and let $F : \mathcal{H} \rightarrow \mathcal{H}$ be a vector field (“operator”). The VI problem asks: *find a point $x^* \in C$ such that F , evaluated at x^* , never points “outward” relative to any other point of C — i.e. moving from x^* towards any other feasible point $y \in C$ never decreases along the $F(x^*)$ direction:*

$$\langle F(x^*), y - x^* \rangle \geq 0 \quad \text{for all } y \in C.$$

Why you should care. VI is a common generalization of two things you already know:

- **Fixed-point problems:** taking $F = \text{Id} - T$ turns the VI exactly into $x^* = T(x^*)$ (shown later this chapter).
- **Constrained optimization:** minimizing a differentiable f over C has first-order optimality condition $\langle \nabla f(x^*), y - x^* \rangle \geq 0$ for all $y \in C$ — precisely the VI with $F = \nabla f$.

The residual — plain language. A **residual** is any quantity that is zero exactly at a solution and measures “how far” a candidate point is from solving the problem, so it can serve as a stopping test or a progress diagnostic. For the fixed-point problem the natural residual is $\|x_n - T(x_n)\|$: it vanishes iff $x_n \in \text{Fix}(T)$. For the equivalent equation $F(x) = 0$ with $F = \text{Id} - T$, the same quantity is $\|F(x_n)\|$.

Three Regimes, One Crucial Choice

Intuition. The relaxation parameter λ_n decides *how big a step* you take from x_n towards $T(x_n)$. Small λ_n : cautious, safe, slow. Large λ_n (overrelaxation, $\lambda_n > 1$): aggressive, can converge much faster — *if* chosen well — but risks divergence if chosen badly.

Definition (Under/Un/Overrelaxation)

The KM iteration is *underrelaxed* if $\lambda_n \leq 1$, *unrelaxed* if $\lambda_n \equiv 1$, *overrelaxed* if $\lambda_n \geq 1$, for all $n \geq 0$.

Why this chapter exists. The convergence *rate* of the KM iteration depends heavily on $\{\lambda_n\}$, yet results on how to select it are limited. The remainder of the chapter surveys: (7.0) known optimal/near-optimal choices from the literature, (7.1) an approximate optimal sequence for general nonexpansive T , (7.2) relaxations inherited from variational inequality projection methods, and (7.3) a residual-driven line search algorithm.

Combettes' Optimal Relaxation Parameter

Setting. Let D be a closed subspace and S a closed convex subset of $\mathcal{H}$, and take $T = P_D P_S$ (compose two metric projections) in the KM iteration. Combettes asked: for a fixed point $p \in \text{Fix}(P_D P_S)$, which λ_n minimizes $\|x_{n+1} - p\|$?

Optimal Relaxation Parameter (7.1)

$$\lambda_n^* = \frac{\langle x_n - P_D P_S(x_n), x_n - p \rangle}{\|x_n - P_D P_S(x_n)\|^2}.$$

- $x_n - P_D P_S(x_n)$ is (minus) the step T would take;
- p is a fixed point — *unknown* in practice, so λ_n^* cannot literally be computed; it is a theoretical benchmark.

Lemma 7.1

The optimal relaxations λ_n^* in (7.1) are always *overrelaxation*: $\lambda_n^* \geq 1$ for every $n \geq 0$. Combettes further claimed λ_n^* could in theory be very large, definitely greater than 2.

Note: optimality *per step* does not automatically guarantee the *whole sequence* converges faster — but for the parallel projection method, overrelaxation was numerically found to accelerate convergence to $\tilde{a}^{/33}$

Approximating λ_n^* : Armijo and Line-Search Ideas

Why approximate? λ_n^* in (7.1) needs the unknown p . Combettes instead used an **Armijo relaxation skill**: shrink λ_n until a sufficient-decrease test on $\Phi(x) = \frac{1}{2} \|x - P_D P_S(x)\|^2$ holds.

Armijo-Type Test (7.2)

$$\Phi(x_n) - \Phi(x_{n+1}) \geq \mu \lambda_n \|\nabla \Phi(x_n)\|^2,$$

where $\mu \in (0, 1)$ is a fixed constant. Caveat: this needs Φ *differentiable*, which fails for $\Phi(x) = \frac{1}{2} \|x - T(x)\|^2$ when T is only a general nonexpansive (or firmly nonexpansive) map.

Xu's line-search viewpoint. Rewrite KM as a descent method: $x_{n+1} = x_n - \lambda_n v_n$ with search direction $v_n = x_n - T(x_n)$. Choose λ_n by solving small 1-D optimization problems:

Xu's Line-Search Choices (7.3)–(7.4)

$$\lambda_n = \operatorname{argmin}_{0 \leq \lambda \leq c} \{ \|x_n(\lambda) - T(x_n)(\lambda)\|^2 - \beta \lambda^2 \|x_n - T(x_n)\| \},$$

$$\lambda_n = \operatorname{argmin}_{0 \leq \lambda \leq 1} \{ \|x_n(\lambda) - T(x_n)(\lambda)\|^2 - \beta \lambda (1 - \lambda) \|x_n - T(x_n)\| \},$$

where $x_n(\lambda) := x_n - \lambda(x_n - T(x_n))$, $c \in (0, 1)$, $\beta > 0$. In practice these 1-D problems are hard to solve.

The Online Relaxation KM Iteration

Idea. Iutzeler and Hendrickx designed a relaxation *update rule* that automatically tunes λ_n using only quantities observable during the run — no line search, no knowledge of a fixed point. Let $T : \mathbb{R}^N \rightarrow \mathbb{R}^N$ be σ -averaged.

Algorithm 3: Online Relaxation KM Iteration

Initialize $\varepsilon \in (0, 2 \min(\sigma, 1 - \sigma))$, x_0 , $x_1 = T(x_0)$, $\lambda_0 = \lambda_1 = 1$. For $n \geq 1$:

$$\lambda_{n+1} = \frac{(2 - \varepsilon)\lambda_n}{2\sigma\lambda_n + 1 - \frac{\lambda_{n-1} \|x_n - x_{n-1}\|}{\lambda_n \|x_{n-1} - x_{n-2}\|}} + \frac{\varepsilon}{4\sigma}, \quad x_{n+1} = (1 - \lambda_{n+1})x_n + \lambda_{n+1}T(x_n).$$

Theorem 7.1 (Iutzeler–Hendrickx)

If $T : \mathbb{R}^N \rightarrow \mathbb{R}^N$ is σ -averaged with $\text{Fix}(T) \neq \emptyset$ for some $\sigma \in (0, 1)$, then $\{x_n\}$ from Algorithm 3 converges to a point in $\text{Fix}(T)$.

Giselsson et al.'s Line-Search Method and the SuperMann Iteration

Two-Step KM With Line-Search Relaxation (7.6)–(7.7)

With $0 < \bar{\lambda} \leq \lambda_{\max}$ fixed:

$$\begin{cases} y_n = (1 - \bar{\lambda})x_n + \bar{\lambda}T(x_n), \\ x_{n+1} = (1 - \lambda_n)y_n + \lambda_nT(y_n), \end{cases} \quad \lambda_n \in [\bar{\lambda}, \lambda_{\max}] \text{ s.t. } \|x_{n+1} - T(x_{n+1})\| \leq (1 - \epsilon) \|y_n - T(y_n)\|.$$

This is a two-step KM iteration: the first-step relaxation is fixed, the second is chosen by a line search testing the fixed-point residual $\|y_n - T(y_n)\|$.

Theorem 7.2

If T is nonexpansive with $\text{Fix}(T) \neq \emptyset$ and $\bar{\lambda} \in (0, 1)$, then $\|x_n - T(x_n)\| \rightarrow 0$ and $\|x_{n+1} - x_n\| \rightarrow 0$.

A cost caveat. Evaluating the residual test costs a full extra evaluation of T — so testing many candidate λ_n 's per iteration may be too expensive unless $T = T_2T_1$ with T_1 affine and T_2 cheap.

SuperMann (Themelis–Patrinos), sketch. Generalizes KM using $R = \text{Id} - T$ and an update direction d_n , with three regimes per step: a *blind* update (trust d_n outright when the residual has shrunk enough), an *educated* update (accept d_n after verifying a residual-decrease safeguard), or a

7.1 Intuition: The Best Step You Could Take (In Hindsight)

Setup. Suppose x_n has already been computed, and take *any* fixed point $p \in \text{Fix}(T)$ arbitrarily. He, Cho, et al. ask the same question Combettes asked for $T = P_D P_S$, but now for a *general* nonexpansive T : which λ_n makes $\|x_{n+1} - p\|$ as small as possible?

Why this is a clean question. For fixed x_n , $T(x_n)$, and p , the quantity $\|x_{n+1} - p\|^2$ is an explicit *quadratic function of the scalar λ_n* — so “best single step” reduces to minimizing a parabola, something we can do exactly with calculus. That is the entire content of the derivation on the next slide.

Where this is going. The resulting formula still involves the unknown p . The rest of Section 7.1: (i) derive the ideal formula, (ii) study its structural properties, (iii) replace the unknown p by *computable* surrogates to get a genuine, implementable algorithm, and (iv) prove that algorithm still converges.

7.1 Derivation: The n -th Optimal Relaxation Parameter

Step 1 — expand the update. Write $x_{n+1} - p = (1 - \lambda_n)(x_n - p) + \lambda_n(T(x_n) - p)$ and expand the squared norm using the standard two-term identity

$$\|(1-t)a + tb\|^2 = (1-t)\|a\|^2 + t\|b\|^2 - t(1-t)\|a-b\|^2:$$

Expansion (7.8)

$$\begin{aligned}\|x_{n+1} - p\|^2 &= (1 - \lambda_n)\|x_n - p\|^2 + \lambda_n\|T(x_n) - p\|^2 - \lambda_n(1 - \lambda_n)\|x_n - T(x_n)\|^2 \\ &= \lambda_n^2\|x_n - T(x_n)\|^2 - \lambda_n(\|x_n - p\|^2 - \|T(x_n) - p\|^2 + \|x_n - T(x_n)\|^2) \\ &\quad + \|x_n - p\|^2.\end{aligned}$$

Step 2 — read off the quadratic in λ_n . This is $A\lambda_n^2 - B\lambda_n + \|x_n - p\|^2$ with $A = \|x_n - T(x_n)\|^2 \geq 0$ and $B = \|x_n - p\|^2 - \|T(x_n) - p\|^2 + \|x_n - T(x_n)\|^2$. Since $A \geq 0$, the minimizer (when $A > 0$) is $\lambda_n = B/(2A)$.

Step 3 — simplify $B/(2A)$. Substituting B and A and splitting the fraction gives the closed form:

The n -th Optimal Relaxation Parameter (7.9)

$$\hat{\lambda}_n = \frac{1}{2} \frac{\|x_n - p\|^2 - \|T(x_n) - p\|^2}{\|x_n - T(x_n)\|^2}$$

7.1 Definitions 7.1 & 7.2: Removing the Dependence on p

Idea. $\hat{\lambda}_{p,n}$ still depends on which fixed point p we picked. To get a single well-defined sequence, take an infimum over all $p \in \text{Fix}(T)$.

Definition 7.1 (Closed Affine C)

Let C be a closed affine set and $T : C \rightarrow C$ nonexpansive. The n -th **optimal relaxation parameter** of the KM iteration is

$$\hat{\lambda}_n = \inf_{p \in \text{Fix}(T)} \hat{\lambda}_{p,n}.$$

Definition 7.2 (General Closed Convex C)

For general closed convex C , $\hat{\lambda}_{p,n}$ can overshoot $[0, 1]$ (or $[0, 1/\sigma]$), which would push x_{n+1} outside C . So cap it:

$$\hat{\lambda}_n = \begin{cases} \min\{\inf_{p \in \text{Fix}(T)} \hat{\lambda}_{p,n}, 1\}, & T \text{ nonexpansive,} \\ \min\{\inf_{p \in \text{Fix}(T)} \hat{\lambda}_{p,n}, 1/\sigma\}, & T \text{ } \sigma\text{-averaged.} \end{cases}$$

These sequences $\{\hat{\lambda}_n\}$ still cannot be computed in practice (they involve $\text{Fix}(T)$), but they let us prove structural facts about how large/small the “true” optimum can be.

7.1 Lemma 7.2: What Do We Know About $\{\hat{\lambda}_n\}$?

Lemma 7.2

Let C be closed convex, $T : C \rightarrow C$, and $\{\hat{\lambda}_n\}$ as in Definition 7.1 or 7.2. Then:

- ❶ If T is nonexpansive, $\hat{\lambda}_n \geq \frac{1}{2}$.
- ❷ If T is σ -averaged, $\hat{\lambda}_n \geq \frac{1}{2\sigma}$ for each $n \geq 0$.
- ❸ If T is *quasi-isometric* ($\|T(x) - p\| = \|x - p\|$ for all $x \in C, p \in \text{Fix}(T)$), then $\hat{\lambda}_n = \frac{1}{2}$ for every n .
- ❹ If $\text{Id} - T$ is injective on C and $(\text{Id} - T)^{-1} : \text{ran}(\text{Id} - T) \rightarrow C$ is L -Lipschitz, then $\hat{\lambda}_n \leq L$ for every n .

Reading the bounds. Part (2) shows the lower bound *grows* as the averaging constant σ shrinks — so to get the largest guaranteed lower bound on the true optimal parameter, one wants the *smallest valid* σ for which T is σ -averaged.

7.1 From Ideal to Practical: The Approximation Formula

The trick. We cannot use p in (7.9) because it is unknown. But $T(x_n)$ and $T(T(x_n))$ are computable, and — if x_n were already close to $\text{Fix}(T)$ — $T(x_n)$ would be a reasonable stand-in for p , and $T(T(x_n))$ a stand-in for $T(p) = p$. So substitute $p \rightarrow T(x_n)$ and $T(x_n) \rightarrow T(T(x_n)) = T^2(x_n)$ directly into (7.9):

The Approximate Optimal Relaxation Parameter (7.12)

$$\lambda_n = \frac{1}{2} + \frac{\|x_n - T(x_n)\|^2 - \|T(x_n) - T^2(x_n)\|^2}{2 \|(x_n - T(x_n)) - (T(x_n) - T^2(x_n))\|^2}, \quad n \geq 0.$$

- $x_n - T(x_n)$: the fixed-point residual at x_n ;
- $T(x_n) - T^2(x_n)$: the fixed-point residual “one step ahead,” at $T(x_n)$;
- the denominator is the norm of the *difference of residuals* — how much the residual changed between x_n and $T(x_n)$.
- Only three operator evaluations are needed ($T(x_n)$, then T of that): fully computable, no oracle for $\text{Fix}(T)$ required.

Numerical experiments in the source paper show (7.12) clearly outperforms both a fixed relaxation parameter and the online relaxation formula (7.5).

7.1 Running Example: Setting Up

Running example (unit-disk projection), recap. Let $\mathcal{H} = \mathbb{R}^2$, $C = \{x : \|x\| \leq 1\}$, $T = P_C$:

$$T(x) = P_C(x) = \begin{cases} x, & \|x\| \leq 1, \\ x/\|x\|, & \|x\| > 1. \end{cases}$$

Take $x_0 = (3, 4)$, so $\|x_0\| = 5$; the fixed point reached along this ray is $x^* = x_0/\|x_0\| = (0.6, 0.8)$.

A convenient structural fact

Whenever $\|x_n\| > 1$, we have $T(x_n) = x_n/\|x_n\| \in C$, hence $T(x_n) \in \text{Fix}(T)$ automatically (any point already inside C is a fixed point of the projection). Consequently $T^2(x_n) = T(T(x_n)) = T(x_n)$: applying T a second time changes nothing. This makes the running example ideal for hand-checking formulas (7.9) and (7.12) exactly, symbolically.

7.1 Running Example: True Optimal vs. Approximate

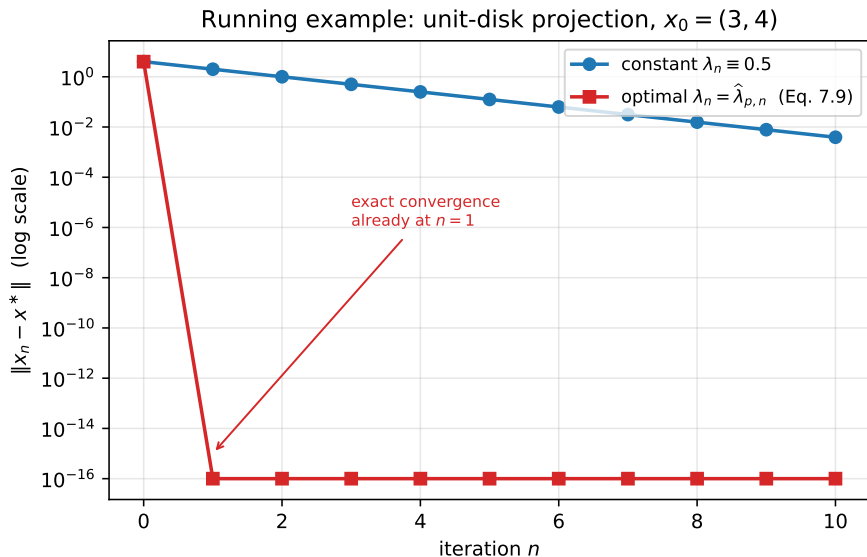
Along the constant-relaxation trajectory ($\lambda_n \equiv 0.5$, which by Chapter 3/5 satisfies $x_n = r_n(0.6, 0.8)$, $r_n = 1 + 4(0.5)^n$), compute *both* formulas at each x_n : the true $\hat{\lambda}_{p,n}$ (7.9) using the *known* $p = x^* = (0.6, 0.8)$, and the practical approximation λ_n (7.12), which uses only $T(x_n)$ and $T^2(x_n)$.

Numerical Comparison

n	x_n	$\hat{\lambda}_{p,n}$ (Eq. 7.9, uses known p)	λ_n (Eq. 7.12, practical)
0	(3.000, 4.000)	1.0	1.0
1	(1.800, 2.400)	1.0	1.0
2	(1.200, 1.600)	1.0	1.0
3	(0.900, 1.200)	1.0	1.0
4	(0.750, 1.000)	1.0	1.0

Reading the table. Because $T(x_n)$ already lies in $\text{Fix}(T)$ whenever $x_n \notin C$ (previous slide), *both* formulas collapse to exactly 1 at every iterate along this trajectory: taking a **full step** ($\lambda_n = 1$, i.e. $x_{n+1} = T(x_n)$) is exactly optimal, and the practical formula (7.12) — with no knowledge of p — *correctly discovers this*. In general, non-degenerate problems, the two formulas need not agree exactly, but this toy case shows why (7.12) is a sensible surrogate: it inherits the same “look one operator application ahead” structure as the true optimal formula.

7.1 Running Example: The Payoff



7.1 Python: Optimal vs. Approximate Relaxation Parameters

```
import numpy as np

def P_C(x):
    """T = metric projection onto the closed unit disk (nonexpansive)."""
    n = np.linalg.norm(x)
    return x if n <= 1.0 else x / n

def lam_hat_p(xn, T, p):
    """Eq. (7.9): n-th optimal relaxation parameter w.r.t. KNOWN p."""
    Txn = T(xn)
    num = np.dot(xn - p, xn - p) - np.dot(Txn - p, Txn - p)
    den = 2 * np.dot(xn - Txn, xn - Txn)
    return 1.0 if den == 0 else 0.5 + num / den

def lam_approx(xn, T):
    """Eq. (7.12): practical approximation, replacing p by T(xn), T(T(xn))."""
    Txn, T2xn = T(xn), T(T(xn))
    num = np.dot(xn - Txn, xn - Txn) - np.dot(Txn - T2xn, Txn - T2xn)
    diff = (xn - Txn) - (Txn - T2xn)
    den = 2 * np.dot(diff, diff)
    return 1.0 if den == 0 else 0.5 + num / den

def km_iteration(x0, lam_fn, T=P_C, n_iter=6):
    xs = [np.asarray(x0, dtype=float)]
    for n in range(n_iter):
        xn = xs[-1]
        lam = lam_fn(n, xn)
```

7.1 Convergence Guarantees for the Approximate Sequence

Theorem 7.3

Let C be closed affine, $T : C \rightarrow C$ nonexpansive with $\text{Fix}(T) \neq \emptyset$. If the optimal $\hat{\lambda}_n$ (7.10) and the approximate λ_n (7.12) satisfy $\lambda_n \leq (2 - \varepsilon)\hat{\lambda}_n$ for some $\varepsilon \in (0, 1)$ and all n , then $\{x_n\}$ generated with $\{\lambda_n\}$ from (7.12) converges weakly to a fixed point.

Theorem 7.4 (Affine Linear Operators)

If $T(x) = Bx + d$ with B symmetric, eigenvalues $\lambda_i \in (-1, 1]$, then using (7.12): (1) $\{x_n\}$ converges to a fixed point; (2) $\|x_n - T(x_n)\| \leq \rho^n \|x_0 - T(x_0)\|$ where $\rho = \sqrt{1 - \left(\frac{1-\bar{\lambda}}{1-\underline{\lambda}}\right)^2}$, $\bar{\lambda} = \max_{i \in J} \lambda_i$, $\underline{\lambda} = \min_{i \in J} \lambda_i$, $J = \{i : \lambda_i \neq 1\}$ — no side condition needed here.

Theorems 7.5–7.6 (Safeguarded Version)

For closed convex C , capping (7.12) with a vanishing safeguard $\{\varepsilon_n\} \subset (0, 1)$, $\sum \varepsilon_n(1 - \varepsilon_n) = \infty$:

$$\lambda_n = \min\left\{(7.12), 1 - \varepsilon_n\right\} \quad (\text{nonexpansive } T), \quad \lambda_n = \min\left\{(7.12), \frac{1}{\sigma}(1 - \varepsilon_n)\right\} \quad (\sigma\text{-averaged}),$$

guarantees weak convergence to a fixed point of T

7.2 Intuition: Fixed Points and Variational Inequalities

Recall (from the Notation Recap). A VI problem asks: find $x^* \in C$ such that F never points “outward” at x^* relative to any other feasible point:

$$\langle F(x^*), y - x^* \rangle \geq 0 \quad \text{for all } y \in C. \quad (7.16)$$

Why bring VI into a chapter on relaxation parameters? Because the fixed-point problem $x^* = T(x^*)$ and the VI problem with $F = \text{Id} - T$ turn out to be *exactly equivalent*. That means: any relaxation-parameter trick invented for VI projection methods (a very active separate research area) can be *imported* into the KM iteration for free, just by setting $F = \text{Id} - T$. This section walks through that equivalence and then imports three such VI algorithms.

Roadmap. (1) State the equivalence precisely. (2) Show $F = \text{Id} - T$ is automatically monotone and Lipschitz whenever T is nonexpansive. (3) Present three modern VI-projection-style algorithms, translated into KM language, each with its own relaxation-parameter rule.

7.2 The Fixed-Point / VI Equivalence

From VI to a projection equation. By a standard characterization of projections (Lemma 2.3 in this book), (7.16) is equivalent to

$$x^* = P_C(x^* - F(x^*)). \quad (7.17)$$

Plug in $F = \text{Id} - T$. If $T : C \rightarrow C$ and $F := \text{Id} - T$, then (7.17) becomes $x^* = P_C(x^* - (x^* - T(x^*))) = P_C(T(x^*))$; since $T(x^*) \in C$, $P_C(T(x^*)) = T(x^*)$, giving

$$x^* = T(x^*). \quad (7.18)$$

Conclusion

The fixed-point problem (7.18) with $T : C \rightarrow C$ and the VI problem (7.16) with $F = \text{Id} - T$ are **equivalent problems**.

Lemma 7.3

If $T : C \rightarrow C$ is nonexpansive, then $F = \text{Id} - T$ is **monotone** ($\langle F(x) - F(y), x - y \rangle \geq 0$) and

2-Lipschitz continuous. *Proof idea:*

$\langle F(x) - F(y), x - y \rangle = \|x - y\|^2 - \langle T(x) - T(y), x - y \rangle \geq \|x - y\| (\|x - y\| - \|T(x) - T(y)\|) \geq 0$ by Cauchy-Schwarz and nonexpansivity; and

$$\|F(x) - F(y)\| = \|(x - y) - (T(x) - T(y))\| \leq \|x - y\| + \|T(x) - T(y)\| \leq 2\|x - y\|.$$

7.2 Algorithm 5: A Tseng-Type Inertial Variant

Where it comes from. Adapting an extragradient-type method for VI (no projection needed here since T already maps $\mathcal{H}$ into itself).

Algorithm 5

Choose $\delta \in (1, \infty)$, $x_0 = y_0 \in \mathcal{H}$, $\lambda_0 > 0$, $\sigma \in (0, \frac{\sqrt{2}-1}{\delta})$. For $n \geq 0$:

$$x_{n+1} = x_n + \lambda_n (T(y_n) - y_n), \quad y_{n+1} = x_{n+1} + \delta(x_{n+1} - x_n),$$

$$\lambda_{n+1} = \begin{cases} \min \left\{ \frac{\sigma \|y_{n+1} - y_n\|}{\|y_{n+1} - y_n - T(y_{n+1}) + T(y_n)\|}, \lambda_n \right\}, & \text{if denominator} \neq 0, \\ \lambda_n, & \text{otherwise.} \end{cases}$$

Reading it as inertial KM. A short calculation shows $x_{n+1} = (1 - \lambda_n)z_n + \lambda_n T(y_n)$ with $z_n = x_n - \delta \frac{\lambda_n}{1 - \lambda_n} (x_n - x_{n-1})$: a *special case of the general inertial KM iteration* (Chapter 6), whose inertial parameter can be *negative* and depends on λ_n itself, while $\delta > 1$ in y_n .

Theorems 7.7–7.8

$\{x_n\}$ converges weakly to a point of $\text{Fix}(T)$ (Thm 7.7). If moreover

$\langle T(x) - T(y), x - y \rangle \leq (1 - M) \|x - y\|^2$ for some $M > 0$, convergence to the unique fixed point is at ^{23/33}

7.2 Algorithm 6: A Second Line-Search Variant

Algorithm 6

Choose $\lambda_0 > 0$, $x_0, y_0 \in \mathcal{H}$, $\mu, \sigma \in (0, 1)$, $\theta \in (0, 1]$, and δ in a specified interval $\subset (\cdot, 1)$. For $n \geq 1$:

$$y_{n+1} = (1 - \delta)x_n + \delta y_n, \quad x_{n+1} = y_{n+1} - \lambda_n x_n + \lambda_n T(x_n),$$

with λ_{n+1} chosen by an adaptive test analogous to Algorithm 5's, built from $\langle x_{n-1} - x_n - T(x_{n-1}) + T(x_n), x_{n+1} - x_n \rangle$.

Structure. Setting $y_0 = x_0$, one finds $y_{n+1} = x_n - \sum_{k=1}^n \delta^{n-k+1}(x_k - x_{k-1})$, so Algorithm 6 is a special case of the **multi-step inertial** KM iteration from Chapter 6 — the inertial parameters may again be negative and involve λ_n ; the relaxation parameters in Algorithms 5 and 6 are both *nonincreasing*.

Theorem 7.9

Let $\mathcal{H}$ be finite-dimensional, T nonexpansive with $\text{Fix}(T) \neq \emptyset$. Then $\{x_n\}$ and $\{y_n\}$ generated by Algorithm 6 converge to the *same* point of $\text{Fix}(T)$.

7.2 Algorithm 7: The Golden Ratio Algorithm

Where it comes from. A golden-ratio-based algorithm for VI, translated to the KM setting. Let $\varphi = \frac{\sqrt{5}+1}{2}$ (golden ratio, $\varphi^2 = 1 + \varphi$).

Algorithm 7

Choose $x_0, x_1 \in \mathcal{H}$, $\lambda_0 > 0$, $\phi \in (1, \varphi]$, $\bar{\lambda} > 0$; set $y_0 = x_1$, $\theta_0 = 1$, $\rho = \frac{1}{\phi} + \frac{1}{\phi^2}$. For each $k \geq 1$:

$$\lambda_n = \min \left\{ \rho \lambda_{n-1}, \frac{\phi \theta_{k-1}}{4 \lambda_{k-1}} \cdot \frac{\|x_n - x_{n-1}\|^2}{\|x_n - x_{n-1} - T(x_n) + T(x_{n-1})\|^2}, \bar{\lambda} \right\},$$

$$y_{n+1} = \left(1 - \frac{1}{\phi}\right)x_n + \frac{1}{\phi}y_n, \quad x_{n+1} = y_{n+1} - \lambda_n x_n + \lambda_n T(x_n), \quad \theta_n = \frac{\lambda_n}{\lambda_{n-1}} \phi.$$

The key difference from Algorithms 5–6. For $\phi \leq \varphi$ we get $\rho \geq 1$, so λ_n can be *larger* than λ_{n-1} — unlike Algorithms 5 and 6, whose relaxation parameters are nonincreasing.

Theorem 7.10

Let $\mathcal{H}$ be finite-dimensional, $T : \mathcal{H} \rightarrow \mathcal{H}$ nonexpansive with $\text{Fix}(T) \neq \emptyset$. Then $\{x_n\}$ from Algorithm 7 converges to a point in $\text{Fix}(T)$.

7.2 Open Problem

Open Problem

How should one choose the *inertial* parameters *and* the *relaxation* parameters of the (multi-step) inertial Krasnosel'skiĭ-Mann iteration *jointly* to accelerate convergence? Results so far treat the two families of parameters somewhat separately; a joint optimal (or near-optimal) selection strategy remains open.

Big picture so far. Algorithms 5, 6, 7 all descend from the *same* equivalence (Lemma 7.3/7.4): take a VI-projection method for $F = \text{Id} - T$, translate the projection step to “apply T ,” and read off whatever relaxation-parameter rule the VI method proposed. This is a template, not a one-off trick — any future VI-projection algorithm can be imported the same way.

7.2 Python: Checking the VI Condition and Algorithm 5

```
import numpy as np

def P_C(x):
    n = np.linalg.norm(x)
    return x if n <= 1.0 else x / n

# --- Geometric check: VI (7.16) holds at  $x^*$  for  $F(x) = x - q$  ---
q, xstar = np.array([3.0, 4.0]), np.array([0.6, 0.8]) #  $xstar = P_C(q)$ 
F_xstar = xstar - q
for y in [np.zeros(2), np.array([-0.8, 0.6]), np.array([1.0, 0.0]), -xstar]:
    print("y=", y, " <F(x*),y-x*> =", np.dot(F_xstar, y - xstar), ">= 0 ?")

# --- Algorithm 5: Tseng-type variant applied to  $T = P_C$  ---
def algorithm5(T, x0, delta=1.5, sigma=0.2, lam0=0.5, n_iter=30):
    xn, yn, lam = x0.copy(), x0.copy(), lam0
    for _ in range(n_iter):
        x_next = xn + lam * (T(yn) - yn)
        y_next = x_next + delta * (x_next - xn)
        num = sigma * np.linalg.norm(y_next - yn)
        den = np.linalg.norm(y_next - yn - T(y_next) + T(yn))
        lam = min(num / den, lam) if den > 1e-14 else lam
        xn, yn = x_next, y_next
    return xn

x0 = np.array([3.0, 4.0])
result = algorithm5(P_C, x0)
print("Algorithm 5 result after 30 iters:", result, " (target (0.6, 0.8))")
```

7.3 Intuition: Steering by the Residual

Recall. The fixed-point problem $x^* = T(x^*)$ is equivalent to the system of nonlinear equations $F(x) = 0$ with $F = \text{Id} - T$:

$$\text{Find } x \in \mathcal{H} \text{ such that } F(x) = 0. \quad (7.28)$$

Idea of this section. Rather than fixing λ_n ahead of time or using a closed-form formula like (7.12), treat λ_n as something to be *searched for at each step*, guided directly by how much the **residual** $\|x_n - T(x_n)\| = \|F(x_n)\|$ would shrink. This borrows directly from derivative-free solvers for nonlinear equations, where a similar residual-driven line search with a **spectral (Barzilai–Borwein-type) coefficient** ρ_n is standard.

What's new here, mechanically. Two moving parts per iteration: (i) a backtracking line search picks a step-size multiplier $\mu_n \in (0, 1]$ so a residual-decrease test holds; (ii) a spectral coefficient ρ_n (updated from consecutive residuals) rescales the step. The overall relaxation parameter is the product $\lambda_n = \rho_n \mu_n$.

7.3 Algorithm 8: The Residual Algorithm (La Cruz)

Algorithm 8

Choose $x_0 \in \mathcal{H}$, $1 \ll \rho_{\max} < \infty$, $\rho_0 \in (0, \rho_{\max}]$, $\gamma, \sigma \in (0, 1)$, and a positive summable sequence $\{\eta_n\}$, $\sum \eta_n = \eta < \infty$. Set $n := 0$, $y_0 = x_0 - T(x_0)$.

- **Step 1:** If $\|x_n - T(x_n)\| = 0$, stop.
- **Step 2:** Find the smallest integer $m_n \geq 0$ such that

$$\|x_n - \sigma^{m_n} \rho_n y_n - T(x_n - \sigma^{m_n} \rho_n y_n)\|^2 \leq (1 - \gamma \sigma^{2m_n}) \|y_n\|^2 + \eta_n. \quad (7.29)$$

Set $\mu_n = \sigma^{m_n}$, $\lambda_n = \rho_n \mu_n$, and

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T(x_n). \quad (7.30)$$

Set $y_{n+1} = x_{n+1} - T(x_{n+1})$ and update the **spectral coefficient**

$$\rho_{n+1} = \begin{cases} \frac{\lambda_n \|y_n\|^2}{\|y_n\|^2 - \langle y_{n+1}, y_n \rangle}, & \text{if } \langle y_{n+1}, y_n \rangle \leq \left(1 - \frac{\rho_n}{\rho_{\max}}\right) \|y_n\|^2, \\ \rho_{\max}, & \text{otherwise.} \end{cases}$$

- **Step 3:** $n := n + 1$; go to Step 1.

Reading it. Algorithm 8 is *exactly* the KM iteration with $\lambda_n = \rho_n \mu_n$: μ_n is found by a line search run *inside* each outer iteration (an auxiliary loop over m), and ρ_n mimics the Barzilai–Borwein spectral stepsize with $\rho_{\max} \gg 1$, chosen so that $\lim_n \|x_n - T(x_n)\| = 0$ (asymptotic regularity) is guaranteed 29 / 33

7.3 Convergence of the Residual Algorithm

Assumption 3.1

$T : \mathcal{H} \rightarrow \mathcal{H}$ is nonexpansive with $\text{Fix}(T) \neq \emptyset$, and the sublevel set

$$\Omega = \{x \in \mathcal{H} : \|x - T(x)\|^2 \leq \|x_0 - T(x_0)\|^2 + \eta\}$$

is **compact**.

Theorem 7.11

If Assumption 3.1 holds, the sequence $\{x_n\}$ generated by Algorithm 8 converges weakly to a point in $\text{Fix}(T)$.

A caveat worth flagging. Assumption 3.1 is fairly restrictive (compactness of a sublevel set in possibly infinite-dimensional $\mathcal{H}$), and the original source does not exhibit an example beyond finite dimensions satisfying it — a reminder that “looks natural” and “easy to verify” are not the same thing.

Cost caveat. Like the line search in Section 7.0’s Giselsson-type method, the inner backtracking loop (7.29) means each outer iteration may cost several evaluations of T , not just one.

7.3 Python: The Residual Algorithm on the Running Example

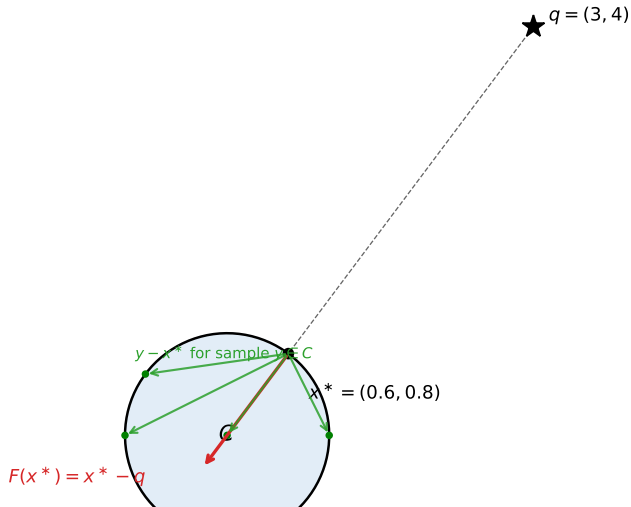
```
import numpy as np

def P_C(x):
    n = np.linalg.norm(x)
    return x if n <= 1.0 else x / n

def residual_algorithm(T, x0, rho_max=10.0, rho0=0.3, gamma=1e-4,
                      sigma=0.5, eta_seq=None, n_iter=15):
    x, y, rho = x0.copy(), x0 - T(x0), rho0
    hist = [x.copy()]
    for n in range(n_iter):
        if np.linalg.norm(x - T(x)) < 1e-12:
            break
        eta_n = eta_seq(n) if eta_seq else 0.0
        m = 0
        while True:
            step = x - (sigma ** m) * rho * y
            lhs = np.linalg.norm(step - T(step)) ** 2
            rhs = (1 - gamma * sigma ** (2 * m)) * np.dot(y, y) + eta_n
            if lhs <= rhs or m > 60:
                break
            m += 1
        lam = rho * sigma ** m
        x_next = (1 - lam) * x + lam * T(x)
        y_next = x_next - T(x_next)
        if np.dot(y_next, y) <= (1 - rho / rho_max) * np.dot(y, y):
            rho = lam * np.dot(y, y) / (np.dot(y, y) - np.dot(y_next, y))
```

7.2 Geometric Picture of a Variational Inequality

Variational inequality: $\langle F(x^*), y - x^* \rangle \geq 0 \quad \forall y \in C$



Chapter 7 Summary

What we learned

- The **optimal relaxation parameter** $\hat{\lambda}_{p,n}$ (Eq. 7.9) minimizes $\|x_{n+1} - p\|$ exactly, but requires the unknown fixed point p — it is a theoretical benchmark, generalizing Combettes' λ_n^* (Eq. 7.1) for $T = P_D P_S$.
- Substituting $T(x_n)$, $T^2(x_n)$ for p gives the **computable approximation** (Eq. 7.12), provably convergent under mild safeguards (Theorems 7.3–7.6).
- The **fixed-point / VI equivalence** ($F = \text{Id} - T$, Lemmas 7.3–7.4) lets any relaxation strategy invented for variational inequality projection methods be imported into KM iterations — illustrated by Algorithms 5, 6, 7 (Tseng-type, line-search, and golden-ratio variants).
- The **residual algorithm** (Algorithm 8) chooses $\lambda_n = \rho_n \mu_n$ via a backtracking line search on the fixed-point residual, combined with a Barzilai–Borwein-style spectral coefficient ρ_n , converging under a (fairly strong) compactness assumption.
- On the **running example** (unit-disk projection, $x_0 = (3, 4)$), the ideal and practical optimal-parameter formulas coincide exactly and reach $x^* = (0.6, 0.8)$ in a single step — versus a slow geometric approach for fixed $\lambda = 0.5$ — illustrating, in the simplest possible setting, why adaptive relaxation matters.